FACULTÉ DES SCIENCES ET TECHNIQUES DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

Atelier de programmation

# Rapport Technique : Structures de Configuration GTK4

Filière d'ingénieur:

**Ingénierie Logicielle et Intégration des Systèmes Informatiques**


Réalisé par :

- El-Assali Saad
- Jounaid Ayoub
- Elmaoui Yahya
- Bichri Yasser

Encadré par :

- Mr. A. Bekkhoucha


Année universitaire : 2026-2027

Ingénierie Logicielle et Intégration des Systèmes Informatiques

# Structures de Configuration GTK4
# Rapport Technique

Projet GTK Wrapper

March 9, 2026

## Contents


**1 Introduction** **4**

**2 Structure `window_config`** **5**
- 2.1 Définition . . . 5
- 2.2 Description des champs . . . 6
- 2.3 Exemple d'utilisation . . . 7

**3 Structure `button_config`** **7**
- 3.1 Définition . . . 7
- 3.2 Description des champs . . . 8
- 3.3 Boutons radio . . . 8
- 3.4 Exemple d'utilisation . . . 9

**4 Structure `widget_style_config`** **9**
- 4.1 Définition . . . 9
- 4.2 Description des champs principaux . . . 10

**5 Structures de Conteneurs (Containers)** **11**
- 5.1 Structure `box_config` . . . 11
- 5.2 Structure `grid_config` . . . 12
- 5.3 Structure `stack_config` . . . 12
- 5.4 Exemple d'utilisation (Box) . . . 13

**6 Éléments d'affichage (Display)** **13**
- 6.1 Structure `label_config` . . . 13
- 6.2 Description des champs (Label) . . . 14
- 6.3 Structure `image_config` . . . 14
- 6.4 Exemple d'utilisation . . . 15
- 6.5 Structure `progress_bar_config` . . . 15
- 6.6 Structure `spinner_config` . . . 16

**7 Éléments de saisie (Input)** **17**
7.1 Structure `entry_config` . . . . . 17
7.2 Description des champs (Entry) . . . . . 17
7.3 Structure `dropdown_config` . . . . . 18
7.4 Exemple d'utilisation . . . . . 19
7.5 Structure `spin_button_config` . . . . . 19

**8 Boutons d'état (Toggle)** **20**
8.1 Structure `checkbox_config` . . . . . 20
8.2 Description des champs (Checkbox) . . . . . 20
8.3 Structure `switch_config` . . . . . 20

**9 Saisie de date (Date)** **21**
9.1 Structure `calendar_config` . . . . . 21
9.2 Description des champs . . . . . 21

**10 Menus et Navigation (Menu)** **22**
10.1 Structures `menu_item_config` et `menubar_config` . . . . . 22
10.2 Description des champs principaux . . . . . 22
10.3 Structure `menu_section_config` . . . . . 23

**11 Boîtes de dialogue (Dialog)** **23**
11.1 Structure `alert_dialog_config` . . . . . 23
11.2 Description des champs . . . . . 24
11.3 Structure `alert_button_config` . . . . . 24

**12 Séparateurs (Separator)** **25**
12.1 Structure `separator_config` . . . . . 25
12.2 Description des champs . . . . . 25

# 1 Introduction

Ce rapport documente les structures de configuration de l'API GTK Wrapper. L'approche utilisée repose sur la configuration déclarative : au lieu d'appeler plusieurs fonctions, le développeur remplit une structure de configuration puis la passe à une fonction de création.

Ce document détaille les structures pour les widgets du wrapper avec une description de chaque champ et des exemples d'utilisation.

# 2 Structure `window_config`

La structure `window_config` permet de configurer une fenêtre GTK4. Elle regroupe les paramètres géométriques, visuels et comportementaux nécessaires à sa création.

## 2.1 Définition

```c
typedef struct {
    const char *title;
    const char *icon_name;
    const char *widget_name;
    const char *background_color;
    const char *background_image_path;
    int default_width;
    int default_height;
    int min_width;
    int min_height;
    int max_width;
    int max_height;
    bool resizable;
    bool decorated;
    bool modal;
    bool maximized;
    bool present_on_create;
    widget_style_config style;
} window_config;
```

## 2.2 Description des champs

| Champ | Type | Description |
|---|---|---|
| title | const char* | Titre affiché dans la barre de titre |
| icon_name | const char* | Icône du thème pour la barre de titre |
| widget_name | const char* | Nom interne pour le débogage |
| background_color | const char* | Couleur de fond (format CSS) |
| background_image_path | const char* | Chemin vers une image de fond |
| default_width | int | Largeur par défaut en pixels |
| default_height | int | Hauteur par défaut en pixels |
| min_width | int | Largeur minimale |
| min_height | int | Hauteur minimale |
| max_width | int | Largeur maximale |
| max_height | int | Hauteur maximale |
| resizable | bool | Autorise le redimensionnement |
| decorated | bool | Affiche la barre de titre |
| modal | bool | Fenêtre modale |
| maximized | bool | État maximisé à la création |
| present_on_create | bool | Affichage immédiat |
| style | widget_style_config | Configuration du style |

Le champ **title** définit le texte affiché dans la barre de titre. Utilisé par les lecteurs d'écran pour l'accessibilité. La valeur peut être NULL.

Le champ **icon_name** spécifie l'icône à afficher. Accepte les noms du thème système ("dialog-information", "document-open"). NULL utilise l'icône par défaut.

Le champ **widget_name** définit un nom interne pour le débogage et les styles CSS. Apparaît dans les outils de développement GTK.

Le champ **background_color** spécifie une couleur de fond au format CSS : noms ("red", "blue"), hexadécimal ("#FF0000") ou RGB ("rgb(255, 0, 0)"). NULL utilise le thème par défaut.

Le champ **background_image_path** definit une image de fond. Le chemin doit pointer vers un fichier image valide. NULL desactive cette fonctionnalite.

Les champs **default_width** et **default_height** définissent les dimensions initiales en pixels.

Les champs **min_width**, **min_height**, **max_width** et **max_height** définissent les contraintes de dimensionnement. 0 désactive la limite.

Le champ **resizable** contrôle si l'utilisateur peut redimensionner la fenêtre. FALSE désactive les poignées de redimensionnement.

Le champ **decorated** contrôle la présence de la barre de titre et des bordures. FALSE crée une fenêtre sans barre de titre.

Le champ **modal** crée une fenêtre modale bloquant l'interaction avec les autres fenêtres jusqu'à sa fermeture.

Le champ **maximized** spécifie si la fenêtre est maximisée dès sa création.

Le champ **present_on_create** contrôle l'affichage immédiat après création.

Le champ **style** est une structure `widget_style_config` pour configurer le style.

### 2.3 Exemple d'utilisation

```c
#include "window.h"

void create_main_window(GtkApplication *app) {
    window_config config = {
        .title = "Mon Application GTK",
        .default_width = 1024,
        .default_height = 768,
        .min_width = 800,
        .min_height = 600,
        .resizable = true,
        .decorated = true,
        .modal = false,
        .maximized = false,
        .present_on_create = true,
        .style = {
            .css_class = "main-window",
            .hexpand = true,
            .vexpand = true
        }
    };

    GtkWidget *window = create_window(app, &config);
    gtk_application_add_window(app, GTK_WINDOW(window));
}
```

## 3 Structure button_config

La structure `button_config` permet de configurer les boutons GTK4. Elle définit le label, l'icône, le style et le comportement callback du bouton.

### 3.1 Définition

```c
typedef struct {
    const char *label;
    const char *icon_name;
    const char *theme_variant;
    bool use_underline;
    bool has_frame;
    bool can_shrink;
    int icon_size;
    widget_style_config style;
    void (*on_clicked)(GtkButton *button, gpointer user_data);
    gpointer user_data;
} button_config;

typedef struct {
    const char *label;
```

```
    GtkWidget *group_with;
    bool is_active;
    widget_style_config style;
    void (*on_toggled)(GtkCheckButton *button, gpointer user_data);
    gpointer user_data;
} radio_button_config;
```

## 3.2 Description des champs

| Champ | Type | Description |
|---|---|---|
| label | const char* | Texte affiché sur le bouton |
| icon_name | const char* | Icône du thème |
| theme_variant | const char* | Variante de thème CSS |
| use_underline | bool | Active les mnémoniques |
| has_frame | bool | Affiche la bordure |
| can_shrink | bool | Autorise la réduction |
| icon_size | int | Taille de l'icône |
| style | widget_style_config | Configuration du style |
| on_clicked | callback | Fonction appelée au clic |
| user_data | gpointer | Données utilisateur |

Le champ **label** définit le texte affiché sur le bouton. Peut contenir des mnémoniques ("_Fichier" pour Alt+F). NULL pour un bouton purement iconique.

Le champ **icon_name** spécifie une icône du thème système ("document-open", "edit-copy", etc.). Peut être combiné avec le label.

Le champ **theme_variant** applique une variante : "primary" (principal), "success" (positif), "danger" (destructif), "secondary" (secondaire).

Le champ **use_underline** active le traitement des mnémoniques clavier.

Le champ **has_frame** contrôle l'affichage de la bordure. FALSE crée un bouton plat.

Le champ **can_shrink** permet au bouton de réduire sa taille minimale.

Le champ **icon_size** définit la taille : GTK_ICON_SIZE_BUTTON (24x24), GTK_ICON_SIZE_ (16x16), etc.

Le champ **style** est une structure `widget_style_config` pour le style.

Le champ **on_clicked** est le callback appelé lors du clic.

Le champ **user_data** transmet des données au callback.

## 3.3 Boutons radio

La structure `radio_button_config` permet de créer des boutons radio pour les selections exclusives.

Le champ **group_with** lie le bouton à un groupe existant.

Le champ **is_active** définit l'état sélectionné initial.

Le champ **on_toggled** est appelé lors du changement d'état.

### 3.4 Exemple d'utilisation

```c
#include "button.h"
#include <stdio.h>

void on_save_clicked(GtkButton *button, gpointer user_data) {
    g_print("Bouton clique!\n");
}

void create_ui_example(GtkWidget *box) {
    button_config save_btn = {
        .label = "_Sauvegarder",
        .theme_variant = "primary",
        .has_frame = true,
        .on_clicked = on_save_clicked,
        .user_data = NULL
    };
    GtkWidget *save_button = create_button(&save_btn);
    gtk_box_append(GTK_BOX(box), save_button);

    radio_button_config opt1 = {
        .label = "Option A",
        .is_active = true
    };
    GtkWidget *radio1 = create_radio_button(&opt1);
    gtk_box_append(GTK_BOX(box), radio1);

    radio_button_config opt2 = {
        .label = "Option B",
        .group_with = radio1
    };
    GtkWidget *radio2 = create_radio_button(&opt2);
    gtk_box_append(GTK_BOX(box), radio2);
}
```

## 4 Structure `widget_style_config`

La structure `widget_style_config` est la fondation visuelle du wrapper. Elle est intégrée dans presque toutes les autres structures pour gérer le CSS, les marges, l'alignement et l'expansion.

### 4.1 Définition

```c
typedef struct {
    const char *css_class;
    const char **css_classes;
    int width_request;
    int height_request;
    int margin_top;
```

```
    int margin_bottom;
    int margin_start;
    int margin_end;
    GtkAlign halign;
    GtkAlign valign;
    bool set_halign;
    bool set_valign;
    bool set_hexpand;
    bool set_vexpand;
    bool set_sensitive;
    bool set_visible;
    bool hexpand;
    bool vexpand;
    bool sensitive;
    bool visible;
    const char *tooltip;
} widget_style_config;
```

## 4.2 Description des champs principaux

| Champ | Type | Description |
|---|---|---|
| css_class(es) | const char* | Classe(s) CSS à appliquer |
| width/height_request | int | Dimensions minimales requises |
| margin_* | int | Marges (haut, bas, début, fin) |
| halign/valign | GtkAlign | Alignement (START, CENTER, END, FILL) |
| hexpand/vexpand | bool | Le widget occupe-t-il l'espace libre ? |
| sensitive | bool | Widget interactif ou grisé |
| visible | bool | Visibilité du widget |
| tooltip | const char* | Texte d'infobulle au survol |

Les champs `css_class` et `css_classes` permettent d'injecter des styles personnalisés via des feuilles de style CSS. Cela permet de décorer un widget (couleurs, bordures, polices) sans modifier le code C, en utilisant les classes standards de GTK ou les vôtres.

Les champs `width_request` et `height_request` définissent la taille minimale que le widget doit occuper. C'est essentiel pour forcer une zone de texte ou un bouton à garder une dimension lisible, même si son contenu est très court.

Les champs `margin_top`, `bottom`, `start` et `end` gèrent la respiration de l'interface. En ajoutant de l'espace autour du widget, vous évitez qu'il ne "colle" aux autres éléments ou aux bords de la fenêtre, ce qui rend l'application plus esthétique.

Les champs `halign` et `valign` (associés à leurs booléens `set_*`) contrôlent l'alignement précis. Ils permettent de placer un widget au centre, au début ou à la fin de l'espace qui lui est alloué, tant horizontalement que verticalement.

Les champs `hexpand` et `vexpand` sont cruciaux pour le redimensionnement. S'ils sont activés, le widget "poussera" les autres pour occuper tout l'espace disponible restant dans la fenêtre, ce qui est typique pour une zone de texte principale ou une liste.

Les champs `sensitive` et `visible` contrôlent l'état interactif. Un widget non sensitif apparaît grisé et ne réagit plus aux clics, tandis qu'un widget non visible disparaît totalement de la mise en page.

Le champ `tooltip` définit le texte d'aide qui s'affiche au survol de la souris. C'est un élément clé de l'accessibilité pour expliquer brièvement la fonction d'un bouton ou d'une icône à l'utilisateur.

# 5 Structures de Conteneurs (Containers)

Les conteneurs permettent d'organiser les widgets dans l'interface. L'API propose des structures pour `GtkBox`, `GtkGrid` et `GtkStack`.

## 5.1 Structure `box_config`

```c
typedef struct {
    GtkOrientation orientation;
    int spacing;
    bool homogeneous;
    widget_style_config style;
} box_config;
```

| Champ | Type | Description |
|---|---|---|
| orientation | GtkOrientation | `GTK_ORIENTATION_HORIZONTAL` ou `VERTICAL` |
| spacing | int | Espace en pixels entre les enfants |
| homogeneous | bool | Si `true`, tous les enfants ont la même taille |
| style | widget_style_config | Configuration de base du widget |

Le champ `orientation` détermine le sens de lecture visuelle de l'interface. En mode `VERTICAL`, les widgets s'empilent comme les éléments d'un menu ou d'un formulaire. En mode `HORIZONTAL`, ils se rangent côte à côte, ce qui est idéal pour créer des barres d'outils ou des rangées de boutons de navigation.

Le champ `spacing` permet de définir un espacement fixe et uniforme entre chaque élément enfant du conteneur. Cela garantit une cohérence visuelle immédiate sans avoir à configurer manuellement les marges individuelles de chaque widget, rendant l'interface plus aérée et lisible.

Le champ `homogeneous` est un outil de design puissant : lorsqu'il est activé, il force tous les widgets enfants à occuper exactement la même largeur (ou hauteur). Cela permet de créer des grilles de boutons parfaitement symétriques, quel que soit le texte qu'ils contiennent, assurant un rendu professionnel et équilibré.

Le champ `style` permet d'appliquer la configuration commune (`widget_style_config`) à l'ensemble du conteneur, facilitant ainsi la gestion des marges globales ou l'application de classes CSS thématiques à tout un groupe de widgets.

## 5.2 Structure grid_config

```
typedef struct {
    int column_spacing;
    int row_spacing;
    bool homogeneous_columns;
    bool homogeneous_rows;
    widget_style_config style;
} grid_config;
```

| Champ | Type | Description |
|---|---|---|
| column/row_spacing | int | Espace entre les colonnes et les lignes |
| homogeneous_* | bool | Force les colonnes/lignes à être de taille égale |

Les champs `column_spacing` et `row_spacing` permettent de créer une grille aérée. Contrairement à une Box, la grille gère l'espace de manière bidimensionnelle, ce qui est crucial pour aligner parfaitement des étiquettes à gauche avec des champs de saisie à droite, comme dans un panneau de configuration.

Les champs `homogeneous_columns` et `homogeneous_rows` garantissent que toutes les cellules de la grille ont la même taille. C'est particulièrement utile pour créer un clavier numérique ou une calculatrice où chaque touche doit avoir des dimensions identiques pour conserver une esthétique régulière.

## 5.3 Structure stack_config

```
typedef struct {
    GtkStackTransitionType transition;
    guint duration_ms;
    bool vhomogeneous;
    bool hhomogeneous;
    widget_style_config style;
} stack_config;
```

| Champ | Type | Description |
|---|---|---|
| transition | GtkStackTransitionType | Type d'animation de transition |
| duration_ms | guint | Durée de l'animation en millisecondes |
| vhomogeneous / hhomogeneous | bool | Force les pages à avoir la même hauteur/largeur |

Le champ `transition` définit l'effet visuel lors du changement de page (glissement latéral, fondu enchaîné, etc.). Cela transforme un changement de contenu brutal en une navigation fluide et moderne, typique des applications mobiles.

Le champ `duration_ms` contrôle la vitesse de cette animation. Une durée courte (environ 300ms) donne une impression de réactivité, tandis qu'une durée plus longue permet à l'utilisateur de bien comprendre la transition entre deux sections de l'application.

Les champs `vhomogeneous` et `hhomogeneous` permettent de décider si le conteneur doit toujours garder la taille de sa plus grande page. Si désactivé, la fenêtre se redimensionnera automatiquement pour s'adapter au contenu de la page actuelle, ce qui offre plus de flexibilité.

### 5.4 Exemple d'utilisation (Box)

```c
#include "container.h"

void create_layout(GtkWidget *parent) {
    box_config my_box = {
        .orientation = GTK_ORIENTATION_VERTICAL,
        .spacing = 10,
        .homogeneous = false,
        .style = { .margin_top = 15, .margin_bottom = 15 }
    };
    GtkWidget *box = create_box(&my_box);
    // gtk_box_append(GTK_BOX(parent), box);
}
```

## 6 Éléments d'affichage (Display)

### 6.1 Structure `label_config`

```c
typedef struct {
    const char *text;
    bool selectable;
    bool wrap;
    PangoEllipsizeMode ellipsize;
    int max_width_chars;
    widget_style_config style;
} label_config;
```

## 6.2 Description des champs (Label)

| Champ | Type | Description |
|---|---|---|
| text | const char* | Texte à afficher |
| selectable | bool | Permet la sélection du texte par l'utilisateur |
| wrap | bool | Retour à la ligne automatique |
| ellipsize | PangoEllipsizeMode | Mode de troncature si le texte est trop long |
| max_width_chars | int | Largeur maximale en caractères |
| style | widget_style_config | Configuration du style |

Le champ `text` constitue le cœur du widget ; il définit la chaîne de caractères qui sera rendue à l'écran. Ce champ supporte les balises Pango, permettant d'appliquer du style (gras, couleur, taille) directement dans le texte.

Le champ `selectable` transforme un texte statique en un élément interactif. Lorsqu'il est activé, l'utilisateur peut surligner et copier le texte (comme un numéro de série ou une URL), ce qui est crucial pour l'accessibilité et l'utilité des informations affichées.

Le champ `wrap` gère la fluidité de l'interface. S'il est activé, le texte passera automatiquement à la ligne au lieu de forcer la fenêtre à s'élargir indéfiniment, permettant ainsi de maintenir une mise en page cohérente sur différents formats d'écran.

Le champ `ellipsize` offre une alternative élégante au retour à la ligne : si l'espace est insuffisant, il tronque le texte en ajoutant des points de suspension au début, au milieu ou à la fin, préservant ainsi l'alignement global.

Le champ `max_width_chars` permet de limiter physiquement la largeur du label en nombre de caractères. Cela aide à définir une structure de page prévisible, empêchant un texte trop long de déformer les conteneurs parents.

Le champ `style` permet d'appliquer la structure `widget_style_config`, gérant ainsi les marges, l'alignement et les classes CSS spécifiques à ce label.

## 6.3 Structure `image_config`

```
typedef struct {
    const char *icon_name;
    const char *file_path;
    int pixel_size;
    bool keep_aspect;
    widget_style_config style;
} image_config;
```

| Champ | Type | Description |
|---|---|---|
| icon_name | const char* | Nom de l'icône du thème système |
| file_path | const char* | Chemin d'accès vers le fichier image |
| pixel_size | int | Taille de l'image en pixels |
| keep_aspect | bool | Conserve les proportions de l'image |

Le champ `icon_name` permet d'utiliser les icônes standards du thème système, garantissant une intégration visuelle parfaite avec l'environnement de bureau de l'utilisateur.

Le champ `file_path` offre la possibilité de charger des images personnalisées (PNG, JPEG, SVG) depuis le disque pour des besoins spécifiques au projet.

Le champ `pixel_size` définit la dimension exacte de l'image. Cela garantit une netteté optimale en évitant les flous liés au redimensionnement automatique.

Le champ `keep_aspect` assure que l'image conserve ses proportions originales, évitant toute déformation visuelle lors des changements de taille du conteneur.

## 6.4 Exemple d'utilisation

```c
#include "display.h"

void create_display_elements(GtkWidget *box) {
    label_config lbl = {
        .text = "Bonjour GTK4",
        .selectable = true,
        .style = { .margin_top = 10 }
    };
    gtk_box_append(GTK_BOX(box), create_label(&lbl));
}
```

## 6.5 Structure progress_bar_config

```c
typedef struct {
    double fraction;
    const char *text;
    GtkOrientation orientation;
    bool show_text;
    bool inverted;
    bool pulse_once;
    widget_style_config style;
} progress_bar_config;
```

| Champ | Type | Description |
|---|---|---|
| fraction | double | Niveau de progression (0.0 à 1.0) |
| text | const char* | Texte superposé sur la barre |
| show_text | bool | Affiche ou masque le texte |
| inverted | bool | Inverse le sens de progression |
| pulse_once | bool | Active le mode indéterminé (va-et-vient) |

Le champ `fraction` définit l'état d'avancement, de 0.0 à 1.0. C'est ce paramètre qui déplace visuellement la barre pour informer l'utilisateur de l'état d'une tâche.

Le champ `text` permet de superposer un message personnalisé ou un pourcentage directement sur la barre de progression.

Le champ `show_text` contrôle la visibilité du texte précédemment défini, permettant d'alterner entre une barre simple et une barre informative.

Le champ `inverted` permet de faire progresser la barre dans le sens inverse (de droite à gauche), offrant plus de flexibilité pour certains designs.

Le champ `pulse_once` active le mode "indéterminé" où un bloc fait des va-et-vient, idéal pour indiquer que le programme travaille sans connaître la durée exacte.

## 6.6 Structure `spinner_config`

```
typedef struct {
    int size;
    bool spinning;
    widget_style_config style;
} spinner_config;
```

| Champ | Type | Description |
|---|---|---|
| size | int | Taille du spinner en pixels |
| spinning | bool | Active ou désactive l'animation |

Le champ `size` permet de définir les dimensions du spinner en pixels. Comme il s'agit d'un indicateur de chargement circulaire, ce paramètre assure que le widget reste proportionnel aux autres éléments de l'interface (comme un bouton ou un label).

Le champ `spinning` agit comme un interrupteur : lorsqu'il est à `true`, l'animation de rotation commence ; à `false`, l'animation s'arrête et le widget devient invisible ou statique. C'est l'outil idéal pour indiquer qu'un processus d'arrière-plan est en cours sans connaître sa durée exacte.

Le champ `style` permet d'appliquer la structure `widget_style_config`. Il est particulièrement utile ici pour gérer l'alignement (`halign`, `valign`) afin de centrer parfaitement l'icône de chargement dans un conteneur ou de lui donner des marges pour qu'il ne colle pas aux bords de la fenêtre.

# 7 Éléments de saisie (Input)

## 7.1 Structure entry_config

```
typedef struct {
    const char *placeholder;
    const char *default_text;
    const char *purpose_hint;
    bool password;
    bool editable;
    int max_length;
    widget_style_config style;
    void (*on_changed)(GtkEditable *editable, gpointer user_data);
    gpointer user_data;
} entry_config;
```

## 7.2 Description des champs (Entry)

| Champ | Type | Description |
|---|---|---|
| placeholder | const char* | Texte indicatif quand le champ est vide |
| default_text | const char* | Texte pré-rempli par défaut |
| password | bool | Masque les caractères saisis |
| editable | bool | Permet la modification par l'utilisateur |
| max_length | int | Longueur maximale de la saisie |
| on_changed | callback | Fonction appelée à chaque modification |

Le champ `placeholder` est essentiel pour l'expérience utilisateur ; il affiche un texte d'aide éphémère à l'intérieur de la zone de saisie, ce qui permet de guider l'utilisateur sans alourdir l'interface avec des étiquettes externes.

Le champ `default_text` permet de pré-remplir le widget avec une valeur existante. C'est une fonctionnalité indispensable pour les formulaires d'édition où l'on souhaite modifier une information déjà enregistrée.

Le champ `password` modifie le mode d'affichage des caractères en les remplaçant par des symboles masqués (généralement des points). C'est la configuration de sécurité standard pour la saisie de mots de passe ou de codes confidentiels.

Le champ `editable` offre un contrôle sur l'état du widget : lorsqu'il est désactivé, l'utilisateur peut toujours lire et copier le contenu, mais toute modification directe du texte est bloquée, transformant ainsi le champ en zone de lecture seule.

Le champ `max_length` définit une contrainte matérielle sur la quantité de données saisies. Cela permet de s'assurer que l'entrée utilisateur respecte les limites de votre base de données ou les contraintes de mémoire de votre application.

Le champ `on_changed` est le point d'entrée pour la programmation événementielle. Il permet d'exécuter du code à chaque fois qu'un caractère est ajouté ou supprimé, ce qui

est idéal pour implémenter une recherche instantanée ou une validation de formulaire en direct.

## 7.3 Structure `dropdown_config`

```
typedef struct {
    const char **options;
    guint selected_index;
    bool enable_search;
    widget_style_config style;
    void (*on_selected)(GtkDropDown *dropdown, GParamSpec *pspec,
        gpointer user_data);
    gpointer user_data;
} dropdown_config;
```

| Champ | Type | Description |
|---|---|---|
| options | const char** | Tableau de chaînes de caractères (choix) |
| selected_index | guint | Index de l'élément sélectionné par défaut |
| enable_search | bool | Active la barre de recherche intégrée |
| on_selected | callback | Fonction appelée lors d'un changement de sélection |

Le champ `options` est un tableau de chaînes de caractères (terminé par `NULL`) qui définit la liste des choix disponibles. C'est l'outil idéal pour restreindre la saisie utilisateur à des valeurs précises, comme une sélection de pays, de thèmes ou de configurations système.

Le champ `selected_index` détermine quelle option est affichée par défaut lors de l'initialisation du widget. Cela permet d'améliorer l'expérience utilisateur en pré-sélectionnant la valeur la plus courante ou celle précédemment enregistrée par l'application.

Le champ `enable_search` active un champ de saisie interne permettant de filtrer les options. Cette fonctionnalité est indispensable lorsque la liste contient de nombreux éléments, car elle évite à l'utilisateur de faire défiler manuellement une longue liste pour trouver son choix.

Le champ `on_selected` est le signal (callback) déclenché dès que l'utilisateur valide un choix. Il permet de récupérer immédiatement l'index sélectionné pour mettre à jour la logique métier ou modifier dynamiquement d'autres parties de l'interface graphique.

Le champ `style` permet d'appliquer la configuration `widget_style_config`. Il est particulièrement utile ici pour définir une largeur minimale (`width_request`) afin que le widget ne change pas de taille de manière inesthétique selon la longueur du texte sélectionné.

## 7.4 Exemple d'utilisation

```c
#include "input.h"

void create_inputs(GtkWidget *box) {
    entry_config entry = {
        .placeholder = "Entrez votre nom...",
        .password = false,
        .editable = true
    };
    gtk_box_append(GTK_BOX(box), create_entry(&entry));
}
```

## 7.5 Structure spin_button_config

```c
typedef struct {
    double min_value;
    double max_value;
    double step;
    double page;
    double value;
    guint digits;
    bool numeric_only;
    bool wrap;
    widget_style_config style;
    void (*on_value_changed)(GtkSpinButton *spin, gpointer
        user_data);
    gpointer user_data;
} spin_button_config;
```

| Champ | Type | Description |
|---|---|---|
| min_value / max_value | double | Limites de la valeur |
| step / page | double | Incréments (flèches et saut de page) |
| value | double | Valeur par défaut |
| digits | guint | Nombre de décimales à afficher |
| numeric_only | bool | Restreint la saisie aux chiffres |
| wrap | bool | Recommence au minimum une fois le maximum atteint |
| on_value_changed | callback | Fonction appelée au changement de valeur |

# 8 Boutons d'état (Toggle)

## 8.1 Structure `checkbox_config`

```
typedef struct {
    const char *label;
    bool active;
    bool inconsistent;
    widget_style_config style;
    void (*on_toggled)(GtkCheckButton *button, gpointer user_data);
    gpointer user_data;
} checkbox_config;
```

## 8.2 Description des champs (Checkbox)

| Champ | Type | Description |
|---|---|---|
| label | const char* | Texte à côté de la case à cocher |
| active | bool | État coché (`true`) ou décoché (`false`) |
| inconsistent | bool | État indéterminé (souvent un tiret) |
| on_toggled | callback | Fonction appelée lors du changement d'état |

Le champ `label` définit le texte descriptif placé à côté de la case à cocher. Il est crucial pour indiquer clairement à l'utilisateur quelle option il est en train d'activer (par exemple : "Accepter les conditions").

Le champ `active` représente l'état booléen de la case. S'il est à `true`, la case apparaît cochée par défaut. C'est l'outil principal pour mémoriser et restaurer les préférences de l'utilisateur.

Le champ `inconsistent` est un état visuel spécial (souvent un tiret) indiquant que l'option n'est ni totalement active, ni totalement inactive. On l'utilise généralement pour des sélections groupées où certains sous-éléments sont cochés et d'autres non.

Le champ `on_toggled` est le signal déclenché à chaque clic. Il permet de mettre à jour instantanément la logique de l'application ou de débloquer d'autres éléments de l'interface suite à une validation.

## 8.3 Structure `switch_config`

```
typedef struct {
    const char *label;
    bool active;
    bool state;
    widget_style_config style;
    void (*on_active_changed)(GtkSwitch *sw, GParamSpec *pspec,
        gpointer user_data);
    gpointer user_data;} switch_config;
```

| Champ | Type | Description |
|---|---|---|
| active | bool | Position de l'interrupteur (On/Off) |
| state | bool | État logique de l'interrupteur |
| on_active_changed | callback | Fonction appelée quand la position change |

Le champ `active` détermine la position physique du commutateur. Contrairement à une case à cocher, le switch est souvent utilisé pour des actions qui ont un effet immédiat et global sur le comportement de l'application.

Le champ `state` définit l'état logique interne. Dans certains cas, il permet de différencier l'état visuel de l'état réel de la fonction (par exemple, si une connexion est en cours).

Le champ `on_active_changed` est le callback dédié à ce widget. Il est utilisé pour déclencher les processus lourds (allumer un module, changer de thème) dès que le commutateur change de position.

# 9 Saisie de date (Date)

## 9.1 Structure `calendar_config`

```
typedef struct {
    GDateTime *selected_date;
    bool show_day_names;
    bool show_week_numbers;
    bool no_month_change;
    widget_style_config style;
    void (*on_day_selected)(GtkCalendar *calendar, gpointer
        user_data);
    gpointer user_data;
} calendar_config;
```

## 9.2 Description des champs

| Champ | Type | Description |
|---|---|---|
| selected_date | GDateTime* | Date sélectionnée par défaut |
| show_day_names | bool | Affiche les en-têtes des jours (Lun, Mar...) |
| show_week_numbers | bool | Affiche les numéros de semaine |
| no_month_change | bool | Empêche l'utilisateur de changer de mois |
| on_day_selected | callback | Fonction appelée lors du clic sur un jour |

Le champ `selected_date` définit la date qui sera mise en évidence lors de l'affichage initial du calendrier. Cela permet de positionner l'utilisateur sur une échéance précise ou sur la date du jour dès l'ouverture du widget.

Le champ `show_day_names` contrôle l'affichage de l'en-tête contenant le nom des jours (Lun, Mar, etc.). Le désactiver permet d'obtenir un rendu plus compact si l'espace est limité.

Le champ `show_week_numbers` ajoute une colonne sur le côté gauche indiquant le numéro de la semaine dans l'année. C'est une fonctionnalité indispensable pour les applications de gestion de projet ou de planification professionnelle.

Le champ `no_month_change` permet de verrouiller le calendrier sur le mois affiché. En activant ce champ, vous empêchez l'utilisateur de naviguer vers d'autres mois, ce qui est utile pour forcer un choix dans une période restreinte.

Le champ `on_day_selected` est le signal déclenché à chaque fois que l'utilisateur clique sur une date. Il permet de récupérer instantanément la nouvelle sélection pour mettre à jour les données de l'application ou fermer une fenêtre de dialogue.

Le champ `style` assure l'intégration visuelle du calendrier (marges, alignement) au sein de votre mise en page globale.

# 10 Menus et Navigation (Menu)

## 10.1 Structures `menu_item_config` et `menubar_config`

```c
typedef struct menu_item_config {
    const char *label;
    menu_item_kind kind;
    const struct menu_item_config *children;
    size_t child_count;
    bool enabled;
    menu_item_callback on_activate;
    gpointer user_data;
} menu_item_config;

typedef struct {
    const menu_section_config *sections;
    size_t section_count;
    GActionMap *action_map;
    menu_layout_mode layout;
    bool show_arrow_indicators;
    widget_style_config style;
} menubar_config;
```

## 10.2 Description des champs principaux

| Champ | Type | Description |
|---|---|---|
| kind | menu_item_kind | Type (Action, Sous-menu, Section) |
| children | menu_item_config* | Sous-éléments du menu |
| on_activate | callback | Fonction appelée au clic |
| layout | menu_layout_mode | Horizontal ou Vertical |

Dans la structure `menu_item_config`, le champ `label` définit le texte visible. Le champ `kind` est crucial : il détermine si l'élément est une action simple, un sous-menu ouvrant d'autres options, ou une section de séparation.

Le champ `children` permet de lier d'autres structures `menu_item_config`, créant ainsi des menus en cascade. Le champ `enabled` permet d'activer ou de griser l'élément selon le contexte, tandis que `on_activate` gère la fonction exécutée au clic.

Pour la `menubar_config`, le champ `sections` organise la barre en blocs logiques (Fichier, Édition). Le champ `layout` définit si la barre est horizontale ou verticale, s'adaptant au design général.

Le champ `show_arrow_indicators` ajoute un indice visuel pour les sous-menus, et le champ `style` assure la cohérence visuelle avec le reste de l'application via les classes CSS.

### 10.3 Structure `menu_section_config`

```c
typedef struct {
    const char *label;
    const menu_item_config *items;
    size_t item_count;
} menu_section_config;
```

| Champ | Type | Description |
|---|---|---|
| label | const char* | Titre de la section (optionnel) |
| items | menu_item_config* | Tableau des éléments du menu pour cette section |
| item_count | size_t | Nombre d'éléments dans la section |

Le champ `label` permet de définir un titre textuel pour un groupe d'éléments au sein d'un menu. Cela est particulièrement utile pour organiser des menus denses en séparant visuellement les catégories d'actions (par exemple, séparer "Édition" de "Formatage").

Le champ `items` est un pointeur vers un tableau de structures `menu_item_config`. Cette approche modulaire permet de construire chaque section indépendamment avant de les assembler dans la barre de menus globale, facilitant ainsi la maintenance du code.

Le champ `item_count` indique au wrapper le nombre exact d'entrées contenues dans la section. Cela permet une allocation mémoire précise et assure que tous les éléments définis sont correctement rendus dans l'interface graphique finale.

## 11 Boîtes de dialogue (Dialog)

### 11.1 Structure `alert_dialog_config`

```c
typedef struct {
    GtkWindow *parent;
    const char *message;
    const char *detail;
    const alert_button_config *buttons;
```

```
    size_t button_count;
    int default_button;
    int cancel_button;
    bool modal;
    void (*on_response)(int response_index, gpointer user_data);
    gpointer user_data;
} alert_dialog_config;
```

## 11.2 Description des champs

| Champ | Type | Description |
|---|---|---|
| parent | GtkWindow* | Fenêtre parente de l'alerte |
| message | const char* | Titre ou message principal |
| detail | const char* | Texte secondaire détaillé |
| buttons | alert_button_config* | Tableau des boutons à afficher |
| on_response | callback | Fonction appelée après un choix |

Le champ `parent` lie l'alerte à une fenêtre principale. Cela permet de centrer le dialogue sur l'application et de s'assurer qu'il reste au premier plan, empêchant l'utilisateur de l'ignorer accidentellement.

Le champ `message` représente le titre en gras de l'alerte. C'est l'information prioritaire qui doit résumer le problème ou la question posée à l'utilisateur de manière concise.

Le champ `detail` offre un espace pour un texte explicatif plus long. On l'utilise pour donner des précisions techniques ou guider l'utilisateur sur la manière de résoudre l'erreur affichée sans surcharger le titre.

Le champ `buttons` est un pointeur vers un tableau de `alert_button_config`. Il permet de définir dynamiquement le nombre et le texte des choix (ex: "Réessayer", "Quitter").

Les champs `default_button` et `cancel_button` utilisent des index pour désigner quel bouton est activé par la touche "Entrée" et lequel l'est par la touche "Échap", améliorant ainsi l'ergonomie au clavier.

Le champ `modal`, lorsqu'il est à `true`, bloque toute interaction avec le reste de l'application tant que le dialogue n'est pas fermé, ce qui est crucial pour les confirmations critiques.

Le champ `on_response` est le callback qui reçoit l'index du bouton cliqué, permettant de brancher la logique de décision suite au choix de l'utilisateur.

## 11.3 Structure `alert_button_config`

```
typedef struct {
    const char *label;
    const char *appearance;
} alert_button_config;
```

| Champ | Type | Description |
|---|---|---|
| label | const char* | Texte du bouton (ex: "OK", "Annuler") |
| appearance | const char* | Style CSS du bouton (ex: "suggested-action") |

Le champ `label` définit le texte qui sera affiché sur le bouton (par exemple : "Accepter", "Annuler" ou "Supprimer"). Il doit être clair et indiquer précisément l'action qui sera déclenchée par le clic de l'utilisateur.

Le champ `appearance` permet d'appliquer un style visuel prédéfini au bouton pour guider l'utilisateur. On utilise généralement la valeur `"suggested"` (souvent coloré en bleu) pour l'action principale recommandée, ou `"destructive"` (coloré en rouge) pour avertir d'une action irréversible comme une suppression.

# 12 Séparateurs (Separator)

## 12.1 Structure `separator_config`

```
typedef struct {
    GtkOrientation orientation;
    widget_style_config style;
} separator_config;
```

## 12.2 Description des champs

| Champ | Type | Description |
|---|---|---|
| orientation | GtkOrientation | `GTK_ORIENTATION_HORIZONTAL` ou `VERTICAL` |
| style | widget_style_config | Configuration (marges, CSS) |

Le champ `orientation` permet de définir si la ligne de séparation doit être horizontale ou verticale (`GTK_ORIENTATION_HORIZONTAL` ou `VERTICAL`). C'est l'outil de design par excellence pour délimiter visuellement deux zones distinctes dans une interface chargée.

On l'utilise couramment pour séparer une barre d'outils du contenu principal ou pour diviser deux colonnes de texte, aidant ainsi l'utilisateur à comprendre la structure hiérarchique de l'application.

Le champ `style` permet d'ajuster l'apparence du séparateur via la structure `widget_style_config`. Il est particulièrement utile pour ajouter des marges (`margin`) afin que la ligne ne touche pas les bords des widgets adjacents, créant une interface plus aérée et professionnelle.